

Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών

Εξάμηνο 7ο : Εργαστήριο Λειτουργικών Συστημάτων Υπολογιστών

1η Εργαστηριακή Άσκηση: Riddle

oslab26
Δημήτριος Θανόγιαννης 03121902
Δημήτριος Καρατζάνος 03121083

Challenge 0:
Εκτελώντας strace στο εκτελέσιμο riddle, παρατηρήσαμε ότι από τα αποτυχημένα system calls υπήρχε μία αποτυχημένη κλήση συστήματος, η

```
openat(AT_FDCWD, ".hello_there", O_RDONLY) = -1 ENOENT (No such file or directory)
```

στην οποία παραπέμπει ο τίτλος της πρόκλησης.
Απλά δημιουργούμε το ζητούμενο αρχείο.
touch .hello_there

Challenge 1:
Παρατηρούμε ότι όταν ξεκινάει το challenge εκτελούνται τα syscalls openat και close για το αρχείο .hello_there. Με βάση το hint του challenge καταλαβαίνουμε ότι πρέπει να αφαιρέσουμε τα δικαιώματα εγγραφής (w) για το αρχείο.
chmod 555 .hello_there

Challenge 2:
Παρατηρούμε χρησιμοποιώντας το strace ότι εκτελούνται 2 syscalls rt_sigaction που ρυθμίζουν τον τρόπο που διαχειρίζεται η διεργασία τα σήματα SIGALRM, SIGCONT. Στην συνέχεια εκτελείται pause με αντίστροφη μέτρηση 10 δευτερολέπτων και τότε στέλνεται το σήμα SIGALRM και σταματάει η διεργασία. Υποψιαζόμαστε ότι πρέπει να στείλουμε το σήμα SIGCONT στην διεργασία ώστε να προχωρήσει σωστά, χρησιμοποιώντας την εντολή:

`kill -CONT $(pidof riddle)` σε 2ο τερματικό.
Πράγματι οι υποψίες μας αποδείχθηκαν πραγματικότητα.

Challenge 3:
Χρησιμοποιώντας την εντολή ltrace παρατηρούμε ότι καλείται η συνάρτηση getenv(ANSWER). Καταλαβαίνουμε ότι πρέπει να ορίσουμε την μεταβλητή περιβάλλοντος ANSWER ίση με την απάντηση στην ερώτηση “What is the answer to life the universe and everything?”, η οποία είναι ο αριθμός 42, με την παρακάτω εντολή.

```
export ANSWER="42"
```

(Θέτουμε επίσης την μεταβλητή περιβάλλοντος I_NEED_TECH_HINTS=1 για επιπλέον hints)

Challenge 4:
Χρησιμοποιώντας την εντολή strace παρατηρούμε ότι καλείται η συνάρτηση open για εγγραφή/ανάγνωση του αρχείου magic_mirror, το οποίο δεν υπάρχει. Επίσης με βάση το tech hint καταλαβαίνουμε ότι το αρχείο αυτό πρέπει να είναι ένα named pipe, οπότε χρησιμοποιούμε την εντολή:

```
mkfifo ./magic_mirror
```

Challenge 5:
Με βάση το αποτέλεσμα της εντολής strace, είδαμε ότι η κλήση συστήματος

```
fcntl(99, F_GETFD)                      = -1 EBADF (Bad file descriptor)
```

απέτυχε λόγω απουσίας αρχείου με fd 99. Έτσι, φτιάξαμε το ακόλουθο πρόγραμμα, το οποίο φτιάχνει ένα αρχείο και ύστερα μέσω της εντολής dup2, του αναθέτει τον fd 99. Στη συνέχεια, για να μπορεί η διεργασία riddle να δει τον fd 99, χρησιμοποιούμε την εντολή execv με το εκτελέσιμο riddle στη συνέχεια του κώδικα.

```c
#include <unistd.h>
#include <fcntl.h>

int main(){
        int fd = open("challenge_5_script", O_RDWR | O_CREAT, 0666);
        dup2(fd, 99);
        close(fd);

        char *args[] = {"./riddle", NULL};
        execv(args[0], args);

        return 0;
}
```

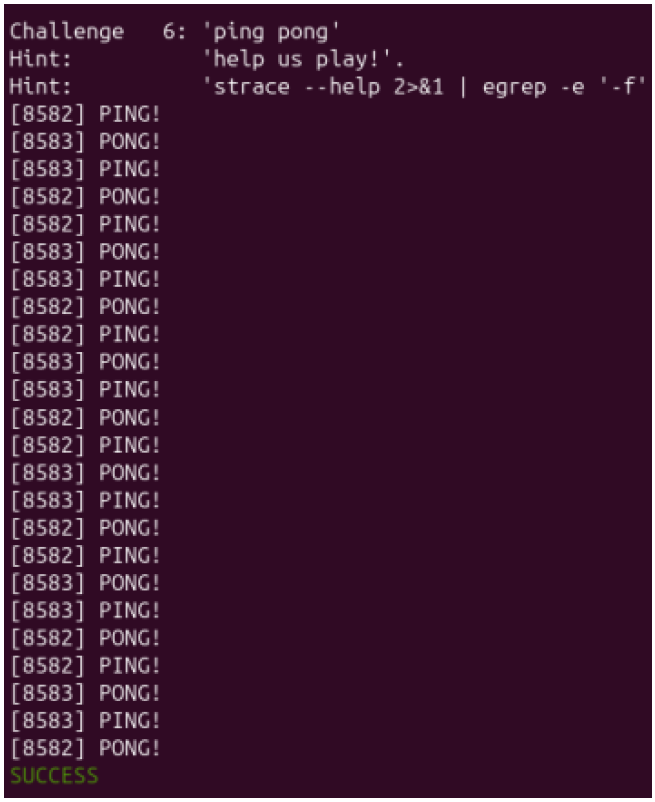
Challenge 6:

Χρησιμοποιούμε την εντολή strace -ff για να καταγράψουμε τα system calls των διεργασιών παιδιών και όχι μόνο του πατέρα. Παρατηρούμε ότι το ένα παιδί θέλει να διαβάσει από το fd 33 και το άλλο να γράψει στο fd 34. Υποψιαζόμαστε ότι χρειάζεται κάποιο pipe για την επικοινωνία των 2 διεργασιών, αφού έχουμε διαφορετικούς fd ανάγνωσης και εγγραφής. Έπειτα παρατηρούμε ότι χρειάζεται και επικοινωνία προς την αντίθετη κατεύθυνση με δεύτερο pipe (fds 53 και 54, σε αντιστοιχία με πριν). Με τον παρακάτω κώδικα C δημιουργούμε τα ζητούμενα pipes και εκτελούμε τον κώδικα του riddle.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    int p[2], p2[2];
    pipe(p);
    pipe(p2);
    dup2(p[0], 33);
    dup2(p[1], 34);
    dup2(p2[0], 53);
    dup2(p2[1], 54);
    system("./riddle");

    return 0;
}
```



Challenge 7:

Χρησιμοποιώντας το strace παρατηρούμε ότι καλούνται 2 system calls lstat για τα αρχεία .hello_there και .hey_there, εκ των οποίων το δεύτερο δεν υπάρχει. Με βάση το hint καταλαβαίνουμε ότι χρειάζεται hard link, οπότε δημιουργούμε το .hey_there ως hard link στο .hello_there :

```
ln .hello_there .hey_there
```

Challenge 8:

Παρατηρούμε ότι η διεργασία προσπαθεί να ανοίξει αρχείο με όνομα bf00 και στην συνέχεια να διαβάσει και να γράψει από την θέση μνήμης 1GB μετά την αρχή του αρχείου, οπότε καταλαβαίνουμε ότι χρειάζεται sparse files. Δημιουργούμε sparse file (μεγέθους 1GB) με την εντολή truncate και γράφουμε κάποια bytes πληροφορίας στην ζητούμενη θέση μνήμης. Κάνουμε το ίδιο για 10 ακόμα αρχεία που απαιτούνται από το πρόγραμμα (bf00-bf10), με την ακόλουθη εντολή:

```
dimitris@debian:~/riddle/riddle-x86-64$ for i in $(seq -w 0 10); do
    touch "bf$i"
    truncate -s 1G "bf$i"
    printf "1" >> "bf$i"
done
```

Challenge 9:
Μέσω του strace, παρατηρούμε ότι η διεργασία ανοίγει ένα socket με πρωτόκολλα επικοινωνίας TCP και IP, και στη συνέχεια επιχειρεί να συνδεθεί με το localhost (127.0.0.1) στη θύρα 49842 με 3-way TCP handshake, το οποίο αποτυγχάνει, αφού επιστρέφεται σημαία RST, δηλαδή ο localhost δεν ακούει σε αυτή τη θύρα. Επομένως, ανοίγουμε μια υπηρεσία σε δεύτερο terminal, όπως παρακάτω:

```
dimitris@debian:~/riddle/riddle-x86-64$ nc -l -p 49842
How much is 5811 + 1? 5812
```

όπου και επιτυγχάνεται η επιθυμητή επικοινωνία.

Challenge 10:
Μέσω του strace, παρατηρήσαμε ότι δημιουργείται ένα αρχείο με όνομα secret_number που περιέχει τον ζητούμενο αριθμό, Στην συνέχεια γίνεται unlink το όνομα του αρχείου, mmap και τέλος κλείνει το αρχείο, άρα και ο fd του.
1ος τρόπος:
Πριν τρέξουμε το riddle φτιάχνουμε το ζητούμενο αρχείο και κάνουμε hard link σε αυτό από ένα άλλο αρχείο. Έτσι μπορούμε να διαβάσουμε το τι γράφτηκε στο secret_number μέσω του άλλου αρχείου παρότι το πρώτο διαγράφηκε.

```
dimitris@debian:~/riddle_exam/riddle-x86-64$ touch secret_number
dimitris@debian:~/riddle_exam/riddle-x86-64$ ln secret_number not_secret_number
dimitris@debian:~/riddle_exam/riddle-x86-64$ cat not_secret_number
The number I am thinking of right now is [uppercase matters]: 2C094CD9
```

2ος τρόπος
Ψάχνουμε στο χάρτη μνήμης τη διεύθυνση του αρχείου:



και αποκτάμε πρόσβαση στο binary του μέσω της εντολής:

```
dimkar@OS-lab-ex1:/proc/10846/map_files$ sudo strings 7609e04a2000-7609e04a3000
The number I am thinking of right now is [uppercase matters]: 6ACAA628
```

όπου μετατρέπει το binary σε strings κατάλληλα για ανάγνωση.

Challenge 11:

Η πρώτη μέθοδος που χρησιμοποιήσαμε στο challenge δεν δουλεύει. Η δεύτερη δουλεύει. Άλλος τρόπος (χωρίς δικαιώματα διαχειριστή και χωρίς hard links) είναι να υλοποιήσουμε μια διεργασία-πατέρα, η οποία δημιουργεί και ανοίγει το αρχείο secret_number και στη συνέχεια εκτελεί fork(). Το παιδί εκτελεί το εκτελέσιμο ./riddle, ενώ ο πατέρας παρακολουθεί και διαβάζει ταυτόχρονα το περιεχόμενο του αρχείου μέσω του δικού του ανοιχτού περιγραφέα (fd). Ο έλεγχος st_nlink που εκτελεί το ./riddle αποτυγχάνει όταν υπάρχουν πολλαπλά ονόματα αρχείου στον δίσκο (hard links). Στη συγκεκριμένη προσέγγιση, ο πατέρας δεν δημιουργεί πρόσθετο hard link, αλλά διατηρεί απλώς έναν ανοιχτό περιγραφέα προς το μοναδικό inode του αρχείου (st_nlink == 1). Έτσι, ακόμη και όταν το ./riddle διαγράψει το αρχείο μέσω unlink(), το inode παραμένει ενεργό στο λειτουργικό σύστημα όσο παραμένει ανοιχτός ο περιγραφέας του πατέρα, επιτρέποντάς του να ανακτήσει το μυστικό. Η ζητούμενη λειτουργία εκτελείται με τον ακόλουθο κώδικα:

```c
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int fd = open("secret_number", O_RDONLY | O_CREAT, 0644);

    if (fork() == 0) {
        char *args[] = {"./riddle", NULL};
        execv(args[0], args);
    }
    else {
        char buffer[100];
        while (1) {
            lseek(fd, 0, SEEK_SET);
            int bytes_read = read(fd, buffer, sizeof(buffer) - 1);

            if (bytes_read > 0) {
                buffer[bytes_read] = '\0';
                printf("Secret is: %s\n", buffer);
            }

            sleep(1);
        }
    }

    return 0;
}
```

Challenge 12:

Μέσω του strace, παρατηρήσαμε ότι δημιουργείται ένα shared memory map ενός αρχείου (riddle-******), και ζητείται να γραφτεί ένας τυχαίος χαρακτήρας, 111 bytes (0x6f) από την αρχική του διεύθυνση. Αυτό επιτυγχάνεται σε δεύτερο terminal με τις ακόλουθες εντολές.

```
dimitris@debian:/tmp$ ls /tmp/
riddle-fHM2oq
ssh-Elgaexo1MC79
systemd-private-99d05c8a653d4cc980ce08f3542729ff-colord.service-kZqOvB
systemd-private-99d05c8a653d4cc980ce08f3542729ff-fwupd.service-TQp7OP
systemd-private-99d05c8a653d4cc980ce08f3542729ff-lm-sensors.service-MNRoOY
systemd-private-99d05c8a653d4cc980ce08f3542729ff-low-memory-monitor.service-bvHrTj
systemd-private-99d05c8a653d4cc980ce08f3542729ff-ModemManager.service-Qq1Exv
systemd-private-99d05c8a653d4cc980ce08f3542729ff-polkit.service-P1oGf7
systemd-private-99d05c8a653d4cc980ce08f3542729ff-power-profiles-daemon.service-5McIWq
systemd-private-99d05c8a653d4cc980ce08f3542729ff-switcheroo-control.service-fPlJed
systemd-private-99d05c8a653d4cc980ce08f3542729ff-systemd-logind.service-SVSbEA
systemd-private-99d05c8a653d4cc980ce08f3542729ff-upower.service-ZORZog
dimitris@debian:/tmp$ echo "J" | dd of="/tmp/riddle-fHM2oq" bs=1 seek=111 count=1 conv=notrun
c status=none
```

Challenge 13:
Εκτελώντας strace, αρχικά παρατηρούμε ότι γίνεται απόπειρα ανοίγματος του αρχείου .hello_there, στο οποίο κανένας χρήστης δεν έχει κανένα δικαίωμα εγγραφής. Αλλάζουμε τα δικαιώματα με chmod, και στη συνέχεια το αρχείο αυτό γίνεται mmap με χωρητικότητα 32 KB, ύστερα γίνεται truncated στα 16 KB και ζητείται να γραφεί χαρακτήρας μετά τα 16 KB, με αποτέλεσμα να στέλνεται το σήμα SIGBUS (εγγραφή σε άκυρη θέση μνήμης)for και να αποτυγχάνει η εκτέλεση της διεργασίας. Επομένως, πρέπει να αυξήσουμε το μέγεθος του αρχείου hello_there, το οποίο γίνεται με την ακόλουθη εντολή.

```
dimitris@debian:~/riddle/riddle-x86-64$ truncate -s 32768 .hello_there
```

Challenge 14:
Από τα Hints καταλαβαίνουμε ότι ζητείται να δοθεί στο riddle το pid 32767. Θα μπορούσαμε να κάνουμε τόσα fork όσα να φτάσουμε το συγκεκριμένο pid, όμως θα κάνουμε κάτι αποδοτικότερο, θα πειράξουμε το τελευταίο pid που ο πυρήνας νομίζει ότι έδωσε σε διεργασία με το 32766, ώστε στη συνέχεια που θα εκτελέσουμε το riddle να πάρει το επιθυμητό pid.

```
dimitris@debian:~/riddle/riddle-x86-64$ echo 32766 | sudo tee /proc/sys/kernel/ns_last_pid
```

TIER 2

Challenge 15:
Καλώντας την strace για την εκτέλεση του challenge, παρατηρούμε ότι αρχικά γίνεται προσπάθεια ανοίγματος του αρχείου tier2.so, το οποίο δεν υπάρχει. Για να δημιουργήσουμε σωστά ένα τέτοιο αρχείο, φτιάχνουμε ένα αρχείο σε C και το μεταγλωττίζουμε σε .so ως εξής:

```
dimitris@debian:~/riddle/riddle-x86-64$ gcc -shared -fPIC -o tier2.so tier2.c
```

Αφού δημιουργηθεί το αρχείο αυτό, το εκτελέσιμο το ανοίγει και προσπαθεί να δεσμεύσει εικονικό χώρο μνήμης για τον κώδικά του, τα read-only data του και για τα read/write data του. Αμέσως μετά, η δέσμευση αυτή εικονικής μνήμης γίνεται unmap. Αν παρατηρήσουμε το αποτέλεσμα της εντολής ltrace, θα δούμε ότι καλείται η συνάρτηση setup_tier2 στο αρχείο, η οποία δεν υπάρχει, και για αυτό και

αποδεσμεύεται η εικονική μνήμη. Φτιάχνουμε επομένως τη ζητούμενη συνάρτηση στο αρχείο tier2.c:

```
void setup_tier2() {}
```

Challenge 16:

Τρέχοντας το εκτελέσιμο, βλέπουμε ότι διακόπτεται βίαια λόγω segmentation fault (σήμα SIGSEGV), το οποίο προκαλείται από την προσπάθεια του εκτελέσιμου να αποκτήσει πρόσβαση στη θέση μνήμης 0x6042000, η οποία, πέρα από το αρχικό page fault, δεν υπάρχει καν στο χάρτη μνήμης της διεργασίας. Εφόσον το εκτελέσιμο καλεί την προηγουμένως δημιουργημένη setup_tier2() συνάρτηση, θα κάνουμε μέσω αυτής mmap για τη ζητούμενη διεύθυνση μνήμης, ώστε να μη συμβαίνει segmentation fault. Αφού δεσμεύσουμε χώρο στην εικονική μνήμη για αυτή τη φυσική θέση μνήμης, το εκτελέσιμο αποτυγχάνει, διότι ζητά να διαβάσει το pid του στη συγκεκριμένη θέση μνήμης, το οποίο και κάνουμε με τον ακόλουθο κώδικα:

```
#include <sys/mman.h>
#include <unistd.h>
#include <sys/types.h>

void setup_tier2(void){

        void *addr = mmap((void *)0x6042000, 4096, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_
ANONYMOUS, -1, 0);

        long int *p = (long int*)addr;

        *p = (long int)getpid();
}
```

TIER 3

Challenge 17:

Τρέχοντας το εκτελέσιμο, βλέπουμε ότι αποτυγχάνει. Το tier3.so υπάρχει ήδη, επομένως δε μπορούμε να το κάνουμε overwrite με ένα δικό μας. Αναλύοντας το αρχείο χρησιμοποιώντας name list βλέπουμε ότι το tier3.so έχει 2 συναρτήσεις:

```
dimkar@OS-lab-ex1:~/Documents/os_lab_ex1/riddle-x86-64-2026-0/riddle-x86-64$ nm -D ./tier3.so
00000000000012a0 T authorize_tier3
                 w __cxa_finalize@GLIBC_2.2.5
                 w __gmon_start__
                 w _ITM_deregisterTMCloneTable
                 w _ITM_registerTMCloneTable
                 U __stack_chk_fail@GLIBC_2.4
0000000000001200 T unlock_tier3
```

Με ltrace βλέπουμε ότι το riddle καλεί την authorize_tier3:

```
dlopen("./tier3.so", 2)                                  = 0x55e6bc5f9950
dlclose(0x55e6bc5f9950)                                  = 0
dlclose(0x55e6bc5f9950)                                  = 0
dlopen("./tier3.so", 2)                                  = 0x55e6bc5f9950
dlsym(0x55e6bc5f9950, "authorize_tier3")                 = 0x7f003f66d2a0
dlclose(0x55e6bc5f9950)                                  = 0
```

Αναλύοντας την assembly του tier3.so μέσω της εντολής:

```
dimitris@debian:~/riddle/riddle-x86-64$ objdump -d -M intel --no-show-raw-insn tier3.so
```

παρατηρούμε τα ακόλουθα 2 σημεία στις συναρτήσεις unlock_tier3() και authorize_tier3(), αντίστοιχα:

```
    1277:       mov    DWORD PTR [rip+0x2daf],eax        # 402c <authorize_tier3@@Base+0x2d8c
>
    127d:       mov    BYTE PTR ds:0xffffffffffffffff,0x1
```

```
    1317:       cmp    DWORD PTR [rip+0x2d0f],eax        # 402c <authorize_tier3@@Base+0x2d8c
>
    131d:       setne  al
```

Η συνάρτηση unlock_tier3() προσπαθεί να γράψει στη θέση μνήμης -1 τον αριθμό 1 και έτσι επιστρέφεται segmentation fault, καθώς η διεύθυνση αυτή ανήκει στο χώρο πυρήνα. Έτσι, ως πρώτο βήμα, φτιάχνουμε έναν signal handler που να χειρίζεται το σήμα SIGSEGV με το να πηγαίνει τον Program Counter 8 θέσεις (Bytes) παρακάτω και να μη σκοτώνει τη διεργασία. Ακόμη και με αυτή την αλλαγή, το tier3 παραμένει κλειδωμένο. Παρατηρώντας τις κλήσεις βιβλιοθηκών μέσω του ltrace, γίνεται 1 φορά άνοιγμα του tier3.so και 2 φορές κλείσιμο του πριν ξανανοιχθεί για την κλήση της συνάρτησης authorize_tier3(), όπου και έπειτα ξανακλείνει. Για να εξασφαλίσουμε ότι μετά τα πολλαπλά κλεισίματα του αρχείου tier3.so θα παραμείνει αναφορά σε αυτό, ώστε να μην αφαιρεθεί από το χάρτη μνήμης ο κώδικάς του, θα πρέπει χειροκίνητα να το ανοίξουμε εμείς τουλάχιστον 2 επιπλέον φορές. Αν κοιτάξουμε τις κλήσεις μέσω της ltrace παραπάνω, θα δούμε ότι παρόλο που τα challenges του tier2 έχουν ολοκληρωθεί, ακόμη καλείται το εκτελέσιμο tier2.so, το οποίο εμείς γράφουμε και έχουμε πρόσβαση:

```
dlopen("./tier2.so", 2)                                 = 0x55e6bc5f7600
dlsym(0x55e6bc5f7600, "setup_tier2")                    = 0x7f003f6721ca
--- SIGSEGV (Segmentation fault) ---
dlclose(0x55e6bc5f7600)                                 = 0
dlopen("./tier3.so", 2)                                 = 0x55e6bc5f9950
dlclose(0x55e6bc5f9950)                                 = 0
dlclose(0x55e6bc5f9950)                                 = 0
dlopen("./tier3.so", 2)                                 = 0x55e6bc5f9950
dlsym(0x55e6bc5f9950, "authorize_tier3")                = 0x7f003f66d2a0
dlclose(0x55e6bc5f9950)                                 = 0
```

Έτσι, στον κώδικα αυτό θα προσθέσουμε τον signal handler και θα ανοίξουμε το tier3.so 3 φορές πριν κλείσει το tier2.so, ώστε να μην παρεμποδιστεί η ομαλή εκτέλεση του tier3.so από τα πολλαπλά κλεισίματα του:

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/mman.h>
#include <dlfcn.h>
#include <signal.h>
#include <ucontext.h>

void sigsegv_handler(int sig, siginfo_t *info, void *context)
{
    ucontext_t *uc = (ucontext_t *)context;

    if ((unsigned long)info->si_addr == 0xffffffffffffffffULL) {
        uc->uc_mcontext.gregs[REG_RIP] += 8;
    }
    else {
        _exit(1);
    }
}
```

```c
void setup_tier2(void){

        void *addr = mmap((void *)0x6042000, 4096, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);

        long int *p = (long int*)addr;

        *p = (long int)getpid();

        struct sigaction sa;
        sa.sa_sigaction = sigsegv_handler;
        sigemptyset(&sa.sa_mask);
        sa.sa_flags = SA_SIGINFO;

        struct sigaction old_sa;
        if (sigaction(SIGSEGV, &sa, &old_sa) < 0) {
                perror("[-] sigaction failed");
                return;
        }

        void *h1 = dlopen("./tier3.so", RTLD_NOW | RTLD_GLOBAL);
        void *h2 = dlopen("./tier3.so", RTLD_NOW | RTLD_GLOBAL);
        void *h3 = dlopen("./tier3.so", RTLD_NOW | RTLD_GLOBAL);

        void (*unlock_tier3)(void) = (void (*)(void))dlsym(h1, "unlock_tier3");

        if (unlock_tier3 != NULL) {
                unlock_tier3();
        }

        sigaction(SIGSEGV, &old_sa, NULL);
}
```

Challenge 18:

Τρέχοντας το εκτελέσιμο, βρισκόμαστε αντιμέτωποι με έναν κωδικό, για τον οποίο δε μας δίνεται καμία πληροφορία, ούτε από την εντολή strace, ούτε από την ltrace, ούτε από κάποιο άλλο αρχείο του εκτελέσιμου. Έτσι, ελέγχουμε μήπως υπάρχει κάποιος άλλος τρόπος να προχωρήσουμε την εκτέλεση, χωρίς να γνωρίζουμε τον κωδικό. Με τις παρακάτω εντολές σε 2ο τερματικό, ελέγχουμε πιθανούς signal handlers που έχει εγκαταστήσει το εκτελέσιμο:

```
dimitris@debian:~/riddle/riddle-x86-64$ pidof riddle
3369
dimitris@debian:~/riddle/riddle-x86-64$ grep -E "Sig(Blk|Ign|Cgt)" /proc/3369/st
atus
SigBlk: 0000000000000000
SigIgn: 0000000000000000
SigCgt: 0000000000400000
```

Παρατηρούμε μη μηδενική τιμή στα signals caught. Αποκωδικοποιώντας τον αριθμό του σήματος, βλέπουμε ότι το 23ο bit του αριθμού είναι ενεργοποιημένο (4 (Hex) = 0100 (Bin)). Παρατηρώντας την έξοδο του strace πριν από το ξεκίνημα των challenges, βλέπουμε την ακόλουθη κλήση συστήματος:

```
rt_sigaction(SIGURG, {sa_handler=0x55671eb043d0, sa_mask=[URG], sa_flags=SA_RESTORER|SA_RESTA
RT, sa_restorer=0x7f617ae40d70}, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
```

η οποία επιβεβαιώνει την υπόθεση μας (το νούμερο του σήματος SIGURG είναι το 23). Επομένως θα στείλουμε στο riddle το συγκεκριμένο σήμα για να δούμε τι θα κάνει:

```
dimitris@debian:~/riddle/riddle-x86-64$ kill -23 3369
```

Στέλνοντας το σήμα αρκετές φορές, ερχόμαστε αντιμέτωποι με τον ακόλουθο μονόλογο:

```
Challenge  18: 'Tier3'
Hint:          'How did you get so far?'.
What's the password?
Eh.. wh.. what?
Who's there?
Leave me alone, I'm just a lowly imp.
STOP POKING ME!
I always wanted to be a dragon slayer...
but for now I'm just trapped in an easter egg :-(
You know, this riddle is IMPossible!
Haha I made a joke!
If I tell you the password will you help me escape?
If you don't know what password, come back when you're a hero
Still here?
OK, the password is 'deep padufly blue'. Please don't go, set me free...
I gave you the password, will you set me free?
I'll tell you my secret...
I'm not really an IMP, I was an ELF once
but I... annoyed the wrong kind of Wizard :-(
```

Τοποθετώντας τον κωδικό που μας δόθηκε, εμφανίζεται το ακόλουθο:

```
Challenge  18: 'Tier3'
Hint:          'How did you get so far?'
What's the password?
deep padufly blue
FAIL
```

Δεν εμφανίζεται η ένδειξη Wrong. , αλλά ακόμη το challenge φαίνεται failed, ενώ αν ξανατρέξουμε το εκτελέσιμο, αποτυγχάνει:

```
dimitris@debian:~/riddle/riddle-x86-64$ ./riddle
bash: ./riddle: cannot execute binary file: Exec format error
```

Το συγκεκριμένο error σημαίνει ότι το εκτελέσιμο δεν έχει έγκυρη κεφαλίδα εκτελέσιμου αρχείου ELF, δηλαδή μετά την εισαγωγή του κωδικού το αρχείο αλλοιώθηκε. Βλέποντας τον τύπο του αρχείου riddle:

```
dimitris@debian:~/riddle/riddle-x86-64$ file ./riddle
./riddle: data
```

Επιβεβαιώνουμε ότι αναγνωρίζεται από το σύστημα ως απλά δεδομένα και όχι εκτελέσιμο.

Παρατηρώντας τα πρώτα 16 Bytes του αρχείου:

```
dimitris@debian:~/riddle/riddle-x86-64$ head -c 16 ./riddle | xxd
00000000: 7f49 4d50 0201 0100 0000 0000 0000 0000  .IMP............
```

βλέπουμε ότι τα πρώτα 4 Bytes, από 0x7f, E, L, F, δηλαδή ο ζητούμενος μορφότυπος των πρώτων Bytes ενός εκτελέσιμου αρχείου, έγιναν 0x7f, I, M, P, ακριβώς όπως συνέβη και με αυτόν που μας μιλούσε πριν!

```
I'm not really an IMP, I was an ELF once
```

Αλλάζουμε έτσι τα 3 αυτά Bytes στη ζητούμενη τους μορφή, ώστε να μπορούμε να τρέξουμε το εκτελέσιμο:

```
dimitris@debian:~/riddle/riddle-x86-64$ echo 'ELF' | dd of=./riddle bs=1 seek=1 count=3 conv=
notrunc
3+0 records in
3+0 records out
3 bytes copied, 8.7432e-05 s, 34.3 kB/s
dimitris@debian:~/riddle/riddle-x86-64$ file ./riddle
./riddle: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), statically linked, stripped
```

Βλέπουμε ότι το riddle ξαναέγινε εκτελέσιμο. Εκτελώντας το:



Μας γίνεται η ερώτηση “Are you a wizard?”, καθώς και εμείς μετατρέψαμε ένα IMP σε ELF!

```
I'm not really an IMP, I was an ELF once
but I... annoyed the wrong kind of Wizard :-(
```